SIMATS ENGINEERING

ASSESSMENT TOOL 1

ANALYTICAL PROBLEM SOLVING

DATABASE MANAGEMENT SYSTEMS (CSA0504)

Submission detail	Information
Course Name	Database Management Systems
Course Code	CSA0504
Course Outcome	CO2 – Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.
Activity Tool	Analytical Problem Solving (High)
Assessment Weightage	40%
Student Name	Pugazharasan K
Registration Number	192511341

Rubric-aligned solution approach

Criterion	How this document addresses it
Problem Understanding	Identifies relations, attributes, keys, conditions, thresholds and required output.
Method Selection	Justifies JOINs, scalar/correlated subqueries, relational algebra, views, indexes and outer joins.
Solution Process	Shows SQL, algebra expressions, intermediate calculations and step-by-step filtering.
Accuracy of Calculation	Computes averages, category totals, balances and exact result rows from the supplied sample data.
Interpretation of Results	Explains why rows qualify or are excluded and discusses performance/security implications.

Source constraints: The assessment PDF specifies hypothetical sample data, MySQL 8.0 syntax, and says not to assume additional attributes. Solutions follow the supplied attributes. Design choices are identified where needed.

QUESTION 1 – JOINS AND SUBQUERIES

Objective: Use Employees and Departments to find Chennai employees, employees above the overall average salary, and employees above their own department's average.

1.1 Relations and method

Relation	Attributes and relationship
Departments	DepartmentID (PK), DepartmentName, Location; one department may have many employees.
Employees	EmpID (PK), Name, DepartmentID (FK), Salary; DepartmentID references Departments.

1.2 Task 1 – INNER JOIN for Chennai departments

```
SELECT e.Name AS EmployeeName, d.DepartmentName, e.Salary
FROM Employees AS e
INNER JOIN Departments AS d ON e.DepartmentID = d.DepartmentID
WHERE d.Location = 'Chennai'
ORDER BY e.EmpID;
```

EmployeeName	DepartmentName	Salary (INR/month)
Arun	IT	60000
Bina	IT	75000
Divya	Finance	80000
Eshan	Finance	55000

Logic: The join matches each employee to their department. IT and Finance are located in Chennai; HR (Bengaluru) and Sales (Mumbai) are excluded.

1.3 Task 2 – Overall average salary subquery

Calculation	Result
Total monthly salaries	60,000 + 75,000 + 45,000 + 80,000 + 55,000 + 50,000 = INR 365,000
Employee count	6
Overall average	365,000 / 6 = INR 60,833.33 approximately

```
SELECT EmpID, Name, DepartmentID, Salary
FROM Employees
WHERE Salary > (SELECT AVG(Salary) FROM Employees)
ORDER BY Salary DESC;
```

EmpID	Name	DepartmentID	Salary (INR/month)
104	Divya	30	80000
102	Bina	10	75000

Interpretation: Divya and Bina are the only employees earning strictly more than INR 60,833.33 per month.

1.4 Task 3 – Correlated subquery

```
SELECT e.EmpID, e.Name, e.DepartmentID, e.Salary
FROM Employees AS e
WHERE e.Salary > (
  SELECT AVG(e2.Salary)
  FROM Employees AS e2
```

```
  WHERE e2.DepartmentID = e.DepartmentID
)
ORDER BY e.DepartmentID, e.Salary DESC;
```

Department	Salary values (INR)	Average	Above-average employee(s)
IT	60,000; 75,000	67,500	Bina – 75,000
HR	45,000	45,000	None; Charan equals average
Finance	80,000; 55,000	67,500	Divya – 80,000
Sales	50,000	50,000	None; Farah equals average

EmpID	Name	DepartmentID	Salary (INR/month)
102	Bina	10	75000
104	Divya	30	80000

Method selection: The first subquery returns one global scalar average. The correlated subquery recalculates the threshold for each employee's department. Because the operator is > rather than >=, equal-to-average salaries do not qualify.

QUESTION 2 – RELATIONAL ALGEBRA WITH MULTIPLE CONDITIONS

Objective: Use selection, projection and joins across Students, Courses and Enrollments.

2.1 Relations

Relation	Attributes / keys
Students	StudentID (PK), StudentName, Programme
Courses	CourseID (PK), CourseName, Credits
Enrollments	StudentID + CourseID composite PK; both are FKs; Marks

2.2 Task 1 – CSE students enrolled in DBMS with marks ≥ 80

R1 ← Students ⋈ Students.StudentID=Enrollments.StudentID Enrollments
R2 ← R1 ⋈ Enrollments.CourseID=Courses.CourseID Courses
R3 ← σ(Programme='CSE' AND CourseName='DBMS' AND Marks≥80)(R2)
Result ← π(StudentName)(R3)

Candidate	Programme	Course	Marks	Decision
Anu	CSE	DBMS	86	Keep: all conditions satisfied
Cynthia	CSE	DBMS	68	Exclude: marks < 80
Bala	IT	DBMS	91	Exclude: not CSE
Deepak	ECE	DBMS	88	Exclude: not CSE

Expected result: StudentName
Anu

2.3 Task 2 – Student and course names for 4-credit courses with marks ≥ 85

R1 ← Enrollments ⋈ Enrollments.StudentID=Students.StudentID Students
R2 ← R1 ⋈ Enrollments.CourseID=Courses.CourseID Courses
R3 ← σ(Credits=4 AND Marks≥85)(R2)
Result ← π(StudentName, CourseName, Marks)(R3)

StudentName	CourseName	Marks
Anu	DBMS	86
Bala	DBMS	91
Deepak	DBMS	88

Why these rows: DBMS and Networks are 4-credit courses. Networks has mark 84 for Cynthia, below the required 85. Python has only 3 credits. The qualifying DBMS marks are 86, 91 and 88.

2.4 Operation sequence

- Join Students to Enrollments using StudentID to attach each mark to the student.
- Join the result to Courses using CourseID to obtain course name and credits.
- Use selection σ to filter rows; all conditions connected by AND must be true.
- Use projection π to return only requested columns.
- Classical relational algebra uses set semantics; SQL normally preserves duplicate rows unless DISTINCT is specified.

Notation: σ = selection (rows), π = projection (columns), ⋈ = join. Explicit join conditions make the expression auditable.

QUESTION 3 – RELATIONAL SCHEMA DESIGN AND UPDATES

Objective: Design Branch, Customer and Account tables, retrieve account details, deposit INR 5,000 and identify customers with multiple accounts.

3.1 Schema and constraints

Table	Primary key	Foreign keys	Purpose
Branch	BranchCode	None	Branch name and city
Customer	CustomerID	None	Customer identity and mobile
Account	AccountNo	CustomerID → Customer; BranchCode → Branch	Account owner, branch, type and balance

Design decisions: use INT for numeric IDs, VARCHAR for codes and names, CHAR(10) for the supplied fixed-length mobile number, and DECIMAL(12,2) for currency to avoid floating-point rounding. Primary/foreign keys preserve identity and referential integrity.

```
CREATE TABLE Branch (
  BranchCode VARCHAR(10) PRIMARY KEY,
  BranchName VARCHAR(80) NOT NULL,
  City VARCHAR(60) NOT NULL
);

CREATE TABLE Customer (
  CustomerID INT PRIMARY KEY,
  CustomerName VARCHAR(100) NOT NULL,
  Mobile CHAR(10) NOT NULL UNIQUE,
  CHECK (Mobile REGEXP '^[0-9]{10}$')
);

CREATE TABLE Account (
  AccountNo INT PRIMARY KEY,
  CustomerID INT NOT NULL,
  BranchCode VARCHAR(10) NOT NULL,
  AccountType VARCHAR(20) NOT NULL,
  Balance DECIMAL(12,2) NOT NULL DEFAULT 0.00,
  CONSTRAINT fk_account_customer FOREIGN KEY (CustomerID)
    REFERENCES Customer(CustomerID),
  CONSTRAINT fk_account_branch FOREIGN KEY (BranchCode)
    REFERENCES Branch(BranchCode),
  CONSTRAINT chk_account_type CHECK (AccountType IN ('Savings','Current')),
  CONSTRAINT chk_account_balance CHECK (Balance >= 0)
);
```

Implementation note: CHECK constraints are enforced in modern MySQL 8.0 releases. Validate phone format in the application as well if the target environment differs. A production banking system would require additional authorization, transaction logging and audit controls beyond this assessment schema.

3.2 Task 2 – Customer, branch, account number and balance

```
SELECT c.CustomerName, b.BranchName, a.AccountNo, a.Balance
FROM Account AS a
INNER JOIN Customer AS c ON a.CustomerID = c.CustomerID
INNER JOIN Branch AS b ON a.BranchCode = b.BranchCode
ORDER BY a.AccountNo;
```

CustomerName	BranchName	AccountNo	Balance (INR)
Kavya	Anna Nagar	5001	30000.00
Manoj	Anna Nagar	5002	45000.00
Kavya	MG Road	5003	15000.00
Nila	MG Road	5004	60000.00

Interpretation: Kavya appears twice because she owns two accounts at different branches; this is valid one-to-many data, not an erroneous duplicate.

3.3 Task 3 – Deposit INR 5,000 into account 5001

```
START TRANSACTION;
UPDATE Account
SET Balance = Balance + 5000.00
WHERE AccountNo = 5001;

SELECT AccountNo, Balance
FROM Account
WHERE AccountNo = 5001;
COMMIT;
```

AccountNo	Opening balance	Deposit	Expected new balance
5001	INR 30,000	INR 5,000	INR 35,000

Why arithmetic update: Balance = Balance + 5000 adds to the stored balance rather than overwriting it with a hard-coded value. In real banking software, authorization and a corresponding transaction record would also be required; the assessment does not define a transaction table.

3.4 Task 4 – Customers holding more than one account

```
SELECT c.CustomerID, c.CustomerName, COUNT(a.AccountNo) AS AccountCount
FROM Customer AS c
INNER JOIN Account AS a ON c.CustomerID = a.CustomerID
GROUP BY c.CustomerID, c.CustomerName
HAVING COUNT(a.AccountNo) > 1
ORDER BY c.CustomerID;
```

CustomerID	CustomerName	AccountCount
201	Kavya	2

Logic: GROUP BY creates a group per customer, COUNT counts accounts and HAVING filters aggregate groups. Kavya owns accounts 5001 and 5003.

QUESTION 4 – TRACE A NESTED SQL QUERY

Objective: Calculate department averages, evaluate each subquery, and show the exact final output ordered by salary descending.

```
SELECT e.Name, e.Salary
FROM Employees e
WHERE e.Salary > (
SELECT AVG(e2.Salary)
FROM Employees e2
  WHERE e2.DepartmentID = e.DepartmentID
)
AND e.DepartmentID IN (
  SELECT DepartmentID FROM Departments
  WHERE DepartmentName IN ('IT', 'HR')
)
ORDER BY e.Salary DESC;
```

4.1 Department average calculations

DepartmentID	Department	Salary values	Calculation	Average (INR)
1	IT	50,000; 70,000; 90,000	190,000 / 3	70,000.00
2	HR	40,000; 60,000	100,000 / 2	50,000.00
3	Sales	55,000	55,000 / 1	55,000.00

4.2 Correlated subquery evaluation

Employee	Department	Salary	Department average	Salary > average?
Asha	IT	50,000	70,000	No
Bharat	IT	70,000	70,000	No – equal
Chitra	IT	90,000	70,000	Yes
Dinesh	HR	40,000	50,000	No
Eva	HR	60,000	50,000	Yes
Farid	Sales	55,000	55,000	No – equal

4.3 IN subquery and final result

```
SELECT DepartmentID FROM Departments
WHERE DepartmentName IN ('IT', 'HR');
-- Allowed DepartmentID values: 1 and 2
```

Name	Salary (INR)
Chitra	90000
Eva	60000

Execution logic: The correlated query compares each salary to the average for that employee's department. The IN query restricts rows to IT and HR. Chitra and Eva satisfy both tests. Asha and Dinesh are below average; Bharat equals the average; Farid equals the Sales average and Sales is not allowed.

Important detail: The operator > excludes employees equal to their department average. The final ORDER BY places Chitra (INR 90,000) before Eva (INR 60,000).

QUESTION 5 – JOIN VERSUS SUBQUERY

Objective: Find Chennai customers who have at least one order above INR 10,000 and compare join/subquery approaches.

5.1 INNER JOIN approach

```
SELECT DISTINCT c.CustomerName
FROM Customers AS c
INNER JOIN Orders AS o ON o.CustomerID = c.CustomerID
WHERE c.City = 'Chennai' AND o.Amount > 10000
ORDER BY c.CustomerName;
```

5.2 EXISTS approach

```
SELECT c.CustomerName
FROM Customers AS c
WHERE c.City = 'Chennai'
AND EXISTS (
   SELECT 1 FROM Orders AS o
   WHERE o.CustomerID = c.CustomerID AND o.Amount > 10000
 )
ORDER BY c.CustomerName;
```

Expected output
Anitha
Chandru

Anitha has a qualifying order for INR 12,000; Chandru has one for INR 15,000. Bobby is in Madurai and Dharani in Coimbatore, so both fail the Chennai condition.

5.3 Evaluation

Aspect	INNER JOIN	EXISTS subquery
Meaning	Combines matching customer and order rows.	Tests whether a qualifying order exists.
Duplicates	A customer can repeat if multiple orders qualify; DISTINCT removes repeats.	Returns each customer row once because EXISTS is a Boolean existence test.
Readability	Good when output needs columns from both relations.	Clear when only existence of a matching order matters.
Performance	Can be efficient with suitable indexes; DISTINCT may need duplicate elimination.	Optimizer may use an efficient semi-join/existence plan; actual plan depends on data and indexes.
Best fit	Correct, but DISTINCT is needed for customer-level output.	Strong fit because the requirement says 'at least one order'.

5.4 Indexes and EXPLAIN

```
CREATE INDEX idx_customers_city_customer
ON Customers (City, CustomerID);

CREATE INDEX idx_orders_customer_amount
ON Orders (CustomerID, Amount);
```

```
EXPLAIN
SELECT c.CustomerName
FROM Customers AS c
WHERE c.City = 'Chennai'
AND EXISTS (
  SELECT 1 FROM Orders AS o
  WHERE o.CustomerID = c.CustomerID AND o.Amount > 10000
);
```

Index rationale: The first index can help locate Chennai customers and their IDs. The second can help find qualifying orders for a customer. Actual usefulness depends on selectivity, row counts, and optimizer decisions.

EXPLAIN: It shows estimated access paths, possible/selected indexes, join order and estimated rows. It does not prove a measured runtime improvement; compare execution plans and test on representative data before making performance claims.

QUESTION 6 – VIEWS AND RESTRICTED ACCESS

Objective: Create a public directory view and an IT salary aggregate view, show outputs and explain privilege controls.

6.1 PublicEmployeeDirectory view

```
CREATE VIEW PublicEmployeeDirectory AS
SELECT e.EmpID, e.EmployeeName, d.DepartmentName, e.Email
FROM Employees AS e
INNER JOIN Departments AS d
ON d.DepartmentID = e.DepartmentID;

SELECT EmpID, EmployeeName, DepartmentName, Email
FROM PublicEmployeeDirectory
ORDER BY EmpID;
```

EmpID	EmployeeName	DepartmentName	Email
101	Arun	IT	arun@example.org
102	Bina	HR	bina@example.org
103	Celine	IT	celine@example.org
104	Dinesh	HR	dinesh@example.org

Security purpose: The view deliberately excludes Salary and BankAccount, exposing only the four requested directory fields.

6.2 ITSalarySummary view

```
CREATE VIEW ITSalarySummary AS
SELECT d.DepartmentName,
    COUNT(e.EmpID) AS EmployeeCount,
    AVG(e.Salary) AS AverageSalary
FROM Departments AS d
INNER JOIN Employees AS e ON e.DepartmentID = d.DepartmentID
WHERE d.DepartmentName = 'IT'
GROUP BY d.DepartmentName;

SELECT DepartmentName, EmployeeCount, AverageSalary
FROM ITSalarySummary;
```

DepartmentName	EmployeeCount	AverageSalary (INR)
IT	2	68500.00

Calculation: (INR 65,000 + INR 72,000) / 2 = INR 68,500. The view exposes an aggregate rather than individual salary values.

6.3 Why a view alone is not a complete security boundary

- A user with direct SELECT permission on Employees may bypass the view and read sensitive columns.
- Use least privilege: grant access to approved views and do not grant direct base-table access to directory readers.
- Check direct grants, role-based privileges and broader database privileges; revoking a single table-level grant may not remove inherited access.

- MySQL view execution depends on SQL SECURITY DEFINER/INVOKER and the privileges of the definer/invoker. Configure and test this deliberately.

```
-- Replace db_name and configure a strong secret securely.
CREATE USER 'directory_reader'@'%' IDENTIFIED BY 'Use-A-Strong-Secret';

GRANT SELECT ON db_name.PublicEmployeeDirectory
TO 'directory_reader'@'%';
GRANT SELECT ON db_name.ITSalarySummary
TO 'directory_reader'@'%';

-- If a direct table-level grant exists, remove it:
REVOKE SELECT ON db_name.Employees
FROM 'directory_reader'@'%';
```

Caveat: REVOKE succeeds only for privileges granted at the stated scope; check roles and broader grants too. The view definer must have required base-table privileges. This is an illustrative privilege design, not a complete production account script.

QUESTION 7 – RELATIONAL ALGEBRA TO SQL

Objective: Return ProductName and Quantity for Accessories products sold in quantities ≥ 4.

7.1 Relational algebra

$R \leftarrow \text{Products} \bowtie \text{Products.ProductID=Sales.ProductID Sales}$

$S \leftarrow \sigma(\text{Category='Accessories' AND Quantity} \geq 4)(R)$

$\text{Result} \leftarrow \pi(\text{ProductName, Quantity})(S)$

7.2 Equivalent SQL

```
SELECT p.ProductName, s.Quantity
FROM Products AS p
INNER JOIN Sales AS s ON s.ProductID = p.ProductID
WHERE p.Category = 'Accessories' AND s.Quantity >= 4
ORDER BY s.SaleID;
```

ProductName	Quantity
Keyboard	5
Webcam	4

Filtering: Keyboard's quantity-5 sale qualifies, but its other sale with quantity 1 does not. Webcam's quantity 4 qualifies. Mouse has quantity 3; Monitor is in Displays rather than Accessories.

7.3 Set versus bag semantics

Feature	Relational algebra	SQL
Duplicate behavior	Classical relations are sets; duplicate tuples are eliminated.	SQL preserves duplicate rows by default unless DISTINCT is specified.
Projection	π returns distinct tuples in classical relational algebra.	SELECT may return repeated product/quantity pairs.
This dataset	Returns Keyboard–5 and Webcam–4.	Same two qualifying rows for the supplied sample; further matching sales could produce repeated rows.

Interpretation: Do not use DISTINCT unless the requirement is one row per unique product/quantity pair. If the aim is to report every qualifying sale, repeated product names may be meaningful.

QUESTION 8 – SQL OPTIMIZATION AND INDEXING

Objective: Rewrite an inefficient query over a production Orders table of 500,000 rows; calculate output from the five sample orders.

8.1 Supplied query pattern

```
SELECT *
FROM Orders
WHERE CustomerID IN (
  SELECT CustomerID FROM Customers WHERE City = 'Chennai'
)
AND YEAR(OrderDate) = 2026
AND MONTH(OrderDate) = 9
ORDER BY TotalAmount DESC;
```

8.2 Optimized rewrite

```
SELECT o.OrderID, o.CustomerID, o.OrderDate, o.TotalAmount
FROM Customers AS c
INNER JOIN Orders AS o ON o.CustomerID = c.CustomerID
WHERE c.City = 'Chennai'
  AND o.OrderDate >= '2026-09-01'
  AND o.OrderDate <  '2026-10-01'
ORDER BY o.TotalAmount DESC;
```

Improvements: Explicit column selection reduces unnecessary transfer; JOIN states the relationship clearly; a half-open date range includes all September dates/times and excludes 1 October onward.

8.3 Index recommendations

```
CREATE INDEX idx_customers_city_customer
ON Customers (City, CustomerID);

CREATE INDEX idx_orders_customer_date
ON Orders (CustomerID, OrderDate);
```

Technique	Why it may help	Trade-off / caution
Customers(City, CustomerID)	Filter Chennai customers and obtain IDs for joining.	Consumes storage and adds write overhead.
Orders(CustomerID, OrderDate)	Supports matching orders to customers and restricting dates.	Column order and benefit depend on selectivity and plan.
Sargable date range	Lets optimizer consider range access on OrderDate without applying functions to each row.	Index use is not guaranteed; optimizer chooses based on estimates.
Explicit columns	Avoids fetching unused columns; can reduce data transfer.	Maintain selected columns as requirements change.

YEAR(OrderDate) and MONTH(OrderDate) apply functions to the indexed column and may prevent an ordinary index range lookup. A direct lower/upper date range is usually more index-friendly. Functional indexes or generated columns are alternatives in some designs.

8.4 Expected output from sample records

OrderID	CustomerID	OrderDate	TotalAmount (INR)
15	4	2026-09-14	18000

12	1	2026-09-10	12000
13	2	2026-09-11	7500

Why these rows: Chennai customers are Kiran (1), Leena (2) and Naveen (4). Order 11 is in August and is excluded; order 14 belongs to Muthu in Salem. The remaining September orders are sorted by TotalAmount descending.

8.5 EXPLAIN-based evaluation

```
EXPLAIN
SELECT o.OrderID, o.CustomerID, o.OrderDate, o.TotalAmount
FROM Customers AS c
INNER JOIN Orders AS o ON o.CustomerID = c.CustomerID
WHERE c.City = 'Chennai'
  AND o.OrderDate >= '2026-09-01'
  AND o.OrderDate < '2026-10-01'
ORDER BY o.TotalAmount DESC;
```

- Inspect access type, possible_keys, key, estimated rows, join order and Extra in EXPLAIN output.
- Check whether the optimizer uses the suggested indexes or chooses a different plan based on statistics.
- ORDER BY TotalAmount may still require a sort; do not assume the proposed indexes remove it.
- Do not claim measured time savings without running comparable tests on representative data and recording the environment.

QUESTION 9 – GROUPING, AGGREGATION AND NESTED QUERIES

Objective: Calculate sales amount by category using Quantity × UnitPrice, then return categories whose total is greater than average category sales.

9.1 GROUP BY and SUM query

```
SELECT p.Category,
      SUM(ol.Quantity * ol.UnitPrice) AS TotalSales
FROM Products AS p
INNER JOIN OrderLines AS ol ON ol.ProductID = p.ProductID
GROUP BY p.Category
ORDER BY TotalSales DESC;
```

9.2 Line-by-line calculations

LineID	Product	Category	Qty	UnitPrice (INR)	Line amount (INR)
1	Pen	Stationery	20	10	20 × 10 = 200
2	Notebook	Stationery	10	60	10 × 60 = 600
3	USB Drive	Electronics	5	500	5 × 500 = 2,500
4	Headphones	Electronics	2	1,200	2 × 1,200 = 2,400
5	Pen	Stationery	30	10	30 × 10 = 300
6	USB Drive	Electronics	1	500	1 × 500 = 500

Category	Calculation	TotalSales (INR)
Electronics	2,500 + 2,400 + 500	5,400
Stationery	200 + 600 + 300	1,100

9.3 Nested query for categories above average

```
SELECT totals.Category, totals.TotalSales
FROM (
  SELECT p.Category, SUM(ol.Quantity * ol.UnitPrice) AS TotalSales
  FROM Products AS p
  INNER JOIN OrderLines AS ol ON ol.ProductID = p.ProductID
  GROUP BY p.Category
) AS totals
WHERE totals.TotalSales > (
  SELECT AVG(category_totals.TotalSales)
  FROM (
    SELECT p2.Category,
          SUM(ol2.Quantity * ol2.UnitPrice) AS TotalSales
    FROM Products AS p2
    INNER JOIN OrderLines AS ol2 ON ol2.ProductID = p2.ProductID
    GROUP BY p2.Category
  ) AS category_totals
)
ORDER BY totals.TotalSales DESC;
```

9.4 Average and final output

Calculation	Value

Electronics total	INR 5,400
Stationery total	INR 1,100
Average of category totals	(5,400 + 1,100) / 2 = INR 3,250
Electronics test	5,400 > 3,250 → qualifies
Stationery test	1,100 > 3,250 → does not qualify

Category	TotalSales (INR)
Electronics	5400

Why the derived table matters: The average must be calculated over category totals, not individual order lines. The inner grouped query creates one total per category; the scalar subquery averages those totals, and the outer query keeps categories above that average.

QUESTION 10 – CHOOSE JOINS, VIEWS OR SUBQUERIES

Objective: Select the best SQL technique for a reusable marks report, DBMS marks above average, and listing all students including those without enrolments.

10.1 Requirement A – Reusable student marks report

Best choice: a VIEW built from INNER JOINs. It centralizes a repeated combination of student, course and enrollment data and provides a consistent report interface.

```
CREATE VIEW StudentMarksReport AS
SELECT s.StudentName, s.Programme, c.CourseName, e.Marks
FROM Students AS s
INNER JOIN Enrollment AS e ON e.StudentID = s.StudentID
INNER JOIN Courses AS c ON c.CourseID = e.CourseID;

SELECT StudentName, Programme, CourseName, Marks
FROM StudentMarksReport
ORDER BY StudentName, CourseName;
```

StudentName	Programme	CourseName	Marks
Anjali	CSE	DBMS	88
Anjali	CSE	Python	92
Basil	CSE	DBMS	65
Chetan	IT	DBMS	91
Chetan	IT	Networks	76
Deepa	IT	Python	85

Why: INNER JOIN is appropriate for a marks report because it requires an actual enrollment record. A view is reusable; it does not automatically materialize results or guarantee security without privilege controls.

10.2 Requirement B – DBMS marks greater than average DBMS marks

```
SELECT s.StudentName, e.Marks
FROM Students AS s
INNER JOIN Enrollment AS e ON e.StudentID = s.StudentID
INNER JOIN Courses AS c ON c.CourseID = e.CourseID
WHERE c.CourseName = 'DBMS'
  AND e.Marks > (
    SELECT AVG(e2.Marks)
    FROM Enrollment AS e2
    INNER JOIN Courses AS c2 ON c2.CourseID = e2.CourseID
    WHERE c2.CourseName = 'DBMS'
  )
ORDER BY e.Marks DESC;
```

DBMS mark calculation	Value
Marks included	88 + 65 + 91 = 244
Number of DBMS enrollments	3
Average	244 / 3 = 81.333…

Expected output: StudentName	Marks
Chetan	91
Anjali	88

Interpretation: Only DBMS marks are included in the average. Chetan (91) and Anjali (88) exceed 81.333…; Basil (65) does not.

10.3 Requirement C – Every student, including those without enrolments

Best choice: LEFT JOIN from Students to Enrollment and Courses. The left table's rows are preserved even if no matching enrollment exists.

```
SELECT s.StudentID, s.StudentName, s.Programme,
    c.CourseName, e.Marks
FROM Students AS s
LEFT JOIN Enrollment AS e ON e.StudentID = s.StudentID
LEFT JOIN Courses AS c ON c.CourseID = e.CourseID
ORDER BY s.StudentID, c.CourseName;
```

StudentID	StudentName	Programme	CourseName	Marks
1	Anjali	CSE	DBMS	88
1	Anjali	CSE	Python	92
2	Basil	CSE	DBMS	65
3	Chetan	IT	DBMS	91
3	Chetan	IT	Networks	76
4	Deepa	IT	Python	85

Expected-output detail: All students in the current sample have at least one enrollment, so the six matching rows contain no NULLs. In a future dataset, a student with no enrollment would still appear, with NULL CourseName and Marks.

10.4 Summary of technique selection

Requirement	Technique	Reason
A – reusable report	VIEW + INNER JOIN	Reusable named query and consistent join logic.
B – DBMS above average	JOIN + scalar subquery	Aggregate threshold is calculated only from DBMS enrollments.
C – every student	LEFT JOIN	Preserves unmatched Students rows.

General rule: Use JOIN when columns from related relations are needed together; a subquery for aggregate comparisons or existence tests; a VIEW for a reusable named query or controlled interface; LEFT JOIN when unmatched rows from the left relation must remain.

CODE OF CONDUCT

I Jagarajupalli Bhanuprakash Reddy/192524171 (Name / Registration Number) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: J.Bhanuprakash Reddy

References and technical basis

1. SIMATS Engineering. Database Management Systems (CSA05), Assessment Tool 1 – Analytical Problem Solving, CO2 / AT1. Supplied assessment PDF; source of all question-specific schemas, sample records and task requirements.
2. MySQL 8.0 Reference Manual: SELECT, joins, subqueries, aggregate functions, CREATE TABLE, constraints, UPDATE, views, GRANT/REVOKE, CREATE INDEX and EXPLAIN.
3. Silberschatz, A., Korth, H. F., and Sudarshan, S. Database System Concepts. Relational model, relational algebra, SQL and query processing.
4. Elmasri, R., and Navathe, S. B. Fundamentals of Database Systems. Relational database design, integrity constraints and query languages.